

MLOps Assignment 2

Name: Mahesh Velakaturi

Roll Number G25AIT2058

Hugging Face Fine-Tuning, Experiment Tracking & Model Deployment

1. Project Links

All the links below are public so that they can be checked by the evaluator.

GitHub: https://github.com/maheshv2058/MLOps_Assignment_2.git

Kaggle: <https://www.kaggle.com/code/maheshvg25ait2058/notebook52300426d0?scriptVersionId=321188517>

Hugging Face Model: <https://huggingface.co/maheshvgv/distilbert-goodreads-genres/tree/main>

W&B Dashboard: <https://wandb.ai/maheshvgv-mahesh/distilbert-goodreads-genres/runs/x2ji9zyu?nw=nwusermaheshvgv>

2. Purpose of the Report and Summary

This report is prepared for submission of assignment-2 of ML Ops. The assignment was primarily focused on giving hands-on experience to build a full MLOps pipeline and learn the complete workflow. It involves tasks like taking a Jupyter notebook, training a model on Kaggle using a free GPU, tracking the experiment using Weights & Biases (W&B), and then pushing the trained model to the Hugging Face Hub.

A DistilBERT model is used which is fine-tuned to predict the genre of a book from its review text. There are 7 genres taken from the UCSD Goodreads review dataset. After training, the model gives an accuracy of 58.5% and a weighted F1 score of 0.585 on the test set.

3. Model Selection Rationale

The distilbert-base-cased model from the Hugging Face Hub is chosen for this assignment. According to its model card, DistilBERT is a smaller and faster version of BERT. When compared to BERT, it keeps about 97% of the language understanding ability but it is about 60% faster and 40% smaller. This helps in getting the training done with limited time and memory (ideal for a free platform like Kaggle).

This is a simple task of text classification and the number of genre classes is 7. Using a very large model would only take more time and GPU memory without giving significant improvement in accuracy. DistilBERT also works very well with the Hugging Face Trainer API, tokenizer, and the Hub, so the full pipeline of data, training, evaluation, and model upload becomes easy and reproducible.

The cased version (i.e., identifying capital and small letters) is used because capital letters in book reviews (like book names and series names) give useful information. For these reasons, DistilBERT gives a good balance between accuracy, speed, and GPU cost, and lets the focus stay on the MLOps workflow.

MLOps Assignment 2

Name: Mahesh Velakaturi

Roll Number G25AIT2058

4. Kaggle Training Setup

Kaggle was used because it gives a free GPU and also has a built-in way to store secret tokens safely.

4.1 Notebook and Environment

A new Kaggle Notebook was created and the project repository was cloned into the /kaggle/working folder at the start of the run to ensure that the notebook always uses the latest code (data.py, train.py, eval.py, utils.py). The required libraries (transformers, datasets, wandb, huggingface_hub, scikit-learn) are installed in the first cell. The Internet toggle was kept ON in the session settings so that packages can be installed and the model can be pushed to Hugging Face.

4.2 GPU Configuration

In the notebook settings, the Accelerator was set to GPU. The notebook used an NVIDIA Tesla T4 GPU. In the code, the model is moved to the GPU using the cuda device. The T4 GPU was enough to train DistilBERT with a batch size of 16 and a maximum sequence length of 512. The final evaluation on 1,400 reviews finished in about 14 seconds.

4.3 Secrets Configuration

API tokens were not written directly in the code. The Hugging Face token (HF_TOKEN) and the W&B API key (WANDB_API_KEY) were added using Kaggle → Add-ons → Secrets. In the notebook, these tokens are read using the UserSecretsClient and then set as environment variables. This keeps the tokens safe and hidden, even when the notebook is made public. The tokens are used for automatic authentication and login to Hugging Face and W&B.

4.4 Training Configuration

Table 1: Experimental Hyperparameters and Setup

Setting	Value
Base model	distilbert-base-cased
Dataset	UCSD Goodreads reviews (7 genres)
Train / test split	5,600 / 1,400 reviews (200 per genre)
Epochs	3
Batch size (train / eval)	16 / 32
Learning rate	3e-5 with 100 warmup steps
Weight decay	0.01

MLOps Assignment 2

Name: Mahesh Velakaturi

Roll Number G25AIT2058

Max sequence length	512
Experiment tracking	Weights & Biases (report_to="wandb")

The Hugging Face Trainer API was used for training. It performs evaluations and saves a checkpoint after every epoch, and `load_best_model_at_end` is set to `True` so the overall best model checkpoint is retained. The `compute_metrics` function returns evaluation accuracy and a weighted F1 score. W&B seamlessly logs the training loss, validation loss, accuracy, F1 score, and all hyperparameters so every run is reproducible.

4.5 W&B Training Dashboard

The training run was fully tracked online on Weights & Biases. Metrics such as train/loss, eval/loss, eval/accuracy, eval/runtime, and train/learning_rate were updated dynamically on the dashboard. The specific run can be verified directly using the run link provided in Section 1.

▼ Config parameters: {} 151 keys _name_or_path: <code>distilbert-base-cased</code> ► accelerator_config: {} 6 keys activation: <code>gelu</code> adam_beta1: <code>0.9</code> adam_beta2: <code>0.999</code> adam_epsilon: <code>0.00000001</code> ▼ architectures: [] 1 item 0: <code>DistilBertForMaskedLM</code> attention_dropout: <code>0.1</code> auto_find_batch_size: <code>false</code> average_tokens_across_devices: <code>true</code> batch_eval_metrics: <code>false</code> batch_size: <code>16</code> bf16: <code>false</code> bf16_full_eval: <code>false</code> bos_token_id: <code>null</code>	▼ Summary metrics: {} 22 keys eval/accuracy: <code>0.586875</code> eval/f1: <code>0.5849877960097578</code> eval/loss: <code>2.2953238487243652</code> eval/runtime: <code>14.0499</code> eval/samples_per_second: <code>113.88</code> eval/steps_per_second: <code>1.779</code> test/accuracy: <code>0.586875</code> test/f1: <code>0.5849877960097578</code> test/loss: <code>2.2953238487243652</code> test/runtime: <code>13.9744</code> test/samples_per_second: <code>114.495</code> test/steps_per_second: <code>1.789</code> total_flos: <code>2,543,646,198,988,800</code> train_loss: <code>2.209989471435547</code> train_runtime: <code>532.702</code> train_samples_per_second: <code>36.043</code>
--	--

MLOps Assignment 2

Name: Mahesh Velakaturi

Roll Number G25AIT2058



5. Results

A final evaluation was performed on the dedicated test set of 1,400 reviews. The overall performance metrics retrieved from the run are structured below.

Table 2: Final Model Evaluation Metrics

Metric	Score
Accuracy	0.585 (58.5%)
F1 Score (weighted)	0.585
Eval Loss	2.197

The achieved classification accuracy is 58.5%, which is substantially higher than a random guessing benchmark (approximately 14.3% for a balanced 7-class task). Upon per-genre performance analysis, distinct genres with specialized, highly indicative vocabularies (such as poetry and comics_graphic) yielded strong localized F1 scores (around 0.81). Conversely, highly overlapping genres such as young_adult and fantasy_paranormal exhibited lower F1 scores due to shared contextual themes and vocabularies. The full classification report was successfully exported as a JSON artifact and uploaded to W&B.

6. Challenges & Learnings

6.1 What Was Hard

- **Large Dataset Management:** The UCSD Goodreads dataset files consist of massive, compressed JSON files. Ingesting, tokenizing, and sampling them within Kaggle's session runtime and physical memory constraints was highly restrictive. To optimize this, initial reviews per genre were parsed, and the resulting structured sample was serialized into a compact pickle file to optimize subsequent execution speed.
- **Overlapping Genre Taxonomies:** A significant portion of books inherently fit into multiple genres simultaneously (multi-label context). However, our classification framework treats this as a single-label target problem. This topological data constraint represents the primary bottleneck preventing a higher baseline accuracy.
- **Pipeline Initialization and Authentication:** Syncing Kaggle Secrets, runtime environment configurations, Hugging Face Hub token authentication, and unique Weights & Biases telemetry instances required multiple iterations. Early runs suffered from ID mismatches, which were rectified to ensure streaming telemetry accurately reached the respective dashboard project.
- **Model Initialization Warnings:** Upon loading the base weights, explicit missing-key and unexpected-key warnings were registered. It required careful investigation to confirm that these notifications are normal occurrences when discarding a pre-trained masked-language head in favor of an uninitialized sequence classification head.

6.2 What Was Learned

- **Structured MLOps Workflows:** Implementing standard engineering practices (secure token managers, version control synchronization, managed cloud-compute nodes, and uniform model distribution centers) guarantees comprehensive end-to-end reproducibility across engineering teams.
- **Value of Integrated Telemetry:** Centralizing loss curves, hardware metrics, and structured evaluation parameters inside a unified UI dashboard (W&B) provides far higher tracking fidelity than volatile notebook cellular printouts.
- **Secret Integrity:** Excluding long-lived infrastructure keys from source control by employing platform-native environment injection tools is non-negotiable prior to transforming repositories into public visibility.
- **Data Bottlenecks over Architecture:** A model's absolute performance upper-bound is structurally dictated by the underlying quality and label alignment of the dataset rather than pure hyperparameter variation.

6.3 What Could Be Done Differently

- Incorporate automated Hyperparameter Tuning sweeps via W&B Sweeps instead of utilizing fixed localized variables.
- Scale up the localized balanced training dataset sample size to build stronger semantic representation for weaker classes.
- Reconfigure the engineering objective into a true multi-label text classification task to map realistic book genre multi-assignments.
- Implement half-precision (fp16) mixed training parameters to save active GPU memory, decrease runtime latency, and unlock quicker iterations.

MLOps Assignment 2

Name: Mahesh Velakaturi

Roll Number G25AIT2058

7. Conclusion

This assignment successfully provided comprehensive exposure to establishing an automated, unified MLOps architecture. A DistilBERT sequence classifier was fine-tuned on cloud GPU infrastructure, protected via encrypted token clients, monitored interactively using W&B, and successfully pushed into production on the Hugging Face Hub. Ultimately, it is evident that reliable engineering workflows, sound secret compliance, and integrated experiment tracking are just as crucial to production machine learning environments as the mathematical models themselves.